SAE-MCVT: A Real-Time and Scalable Multi-Camera Vehicle Tracking Framework Powered by Edge Computing

Yuqiang Lin[a], Sam Lockyer[a], Florian Stanek[b], Markus Zarbock[b], Adrian Evans[a], Wenbin Li[a], Nic Zhang[a,*]

[a] *University of Bath, United Kingdom*
[b] *Starwit Technologies GmbH, Germany*

arXiv:2511.13904v1 [cs.CV] 17 Nov 2025

Abstract

In modern Intelligent Transportation Systems (ITS), cameras are a key component due to their ability to provide valuable information for multiple stakeholders. A central task is Multi-Camera Vehicle Tracking (MCVT), which generates vehicle trajectories and enables applications such as anomaly detection, traffic density estimation, and suspect vehicle tracking. However, most existing studies on MCVT emphasize accuracy while overlooking real-time performance and scalability. These two aspects are essential for real-world deployment and become increasingly challenging in city-scale applications as the number of cameras grows. To address this issue, we propose SAE-MCVT, the first scalable real-time MCVT framework. The system includes serval edge devices that interact with one central workstation separately. On the edge side, live RTSP video streams are serialized and processed through modules including object detection, object tracking, geo-mapping, and feature extraction. Only lightweight metadata—vehicle locations and deep appearance features—are transmitted to the central workstation. On the central side, cross-camera association is calculated under the constraint of spatial–temporal relations between adjacent cameras, which are learned through a self-supervised camera link model. Experiments on the RoundaboutHD dataset show that SAE-MCVT maintains real-time operation on 2K 15 FPS video streams and achieves an IDF1 score of 61.2. To the best of our knowledge, this is the first scalable real-time MCVT framework suitable for city-scale deployment.

Keywords: Multi-Camera Vehicle Tracking, Edge Computing, Computer Vision, Smart Cities, Traffic monitoring

1. Introduction

The rapid development of urbanization has posed new challenges for transportation authorities in maintaining efficient, safe, and sustainable traffic operations. As road networks become more complex and the number of vehicles continues to grow, conventional traffic management methods are often insufficient to handle growing transportation demands. To address these challenges, leveraging existing overhead traffic cameras has become a practical and cost-effective approach. These cameras continuously capture rich visual information about road environments that can be processed through advance computer vision technologies and provided data-driven insights for intelligent traffic management.

Among these techniques, multi-camera vehicle tracking (MCVT) has emerged as a particularly promising approach. The main goal of MCVT is to associate vehicle trajectories across different cameras, ensuring consistent vehicle identities between different field of view (FOV) (Amosa et al., 2023). This capability enables the reconstruction of complete vehicle trajectories across the monitored area. Once it applies to multiple vehicles, MCVT can effectively generate a comprehensive representation of the traffic flow. With such reconstructed traffic flow data, a variety of downstream applications can be achieved, including traffic density estimation, suspect vehicle retrieval, traffic anomaly detection and so on. Collectively, these applications can be used as components to build a safer, smarter, and more efficient transportation system.

Generally, the MCVT problem can be divided into two stages (Holla et al., 2025): **(i) the intra-camera process**, which detects and tracks vehicles within the same camera view across consecutive frames; and **(ii) the inter-camera**

*Corresponding author.

process, which associates vehicle identities across different camera FOVs. A major challenge in MCVT arises from the small inter-class variability and large intra-class variability among vehicle identities. The same vehicle can appear dramatically different under varying lighting conditions, viewpoints, or occlusions, which complicates cross-camera matching. Conversely, vehicles of the same model often exhibit only subtle appearance differences, making re-identification difficult even for humans(Tang et al., 2019). Luckily, in recent years, many researchers have developed different methods to address those challenges e.g.(Liu et al., 2021; Qian et al., 2020; Yang et al., 2022), achieving promising accuracy and establishing a solid theoretical foundation for the MCVT task. However, most of existing approaches rely on complex deep neural network or multi-stage algorithmic pipelines—-such as performing heavy offline re-matching after single-camera tracking or combining outputs from multiple tracking algorithms (Li et al., 2022). Those designs take high computational cost and restrict these methods to offline operation. Moreover, when the system scales to a city-wide deployment with numerous cameras, the computational burden increases rapidly. For the intra-camera process, it increase linearly but the inter-camera identity association will increase quadratically. This inherent growth significantly limits both the **real-time performance and the scalability** of existing MCVT systems, resulting in a persistent gap between academic research and real-world implementation.

With the rapid advancement of mobile hardware and the growing adoption of edge computing technologies, real-time and scalable solutions for multi-camera vehicle tracking (MCVT) are becoming increasingly feasible (Gong et al., 2023; Ke et al., 2020). Generally, edge computing enables the distribution of computational tasks from a central server to devices located closer to the data source. In the context of MCVT, using this edge-based architecture allows the entire intra-camera process to be deployed and executed at the computational unit near camera(Ke et al., 2022). To be specific, each edge node independently conducts the object detection and single camera tracking tasks, and only the processed metadata—such as vehicle IDs, appearance features, Bounding box location and timestamps—are transmitted to the central server for the subsequent inter-camera association stage(Zhou et al., 2021). This distributed design offers several advantages: **(i)** it significantly reduces the computational reliance on the central server, as most of the processing is completed in parallel across edge nodes, improving overall system scalability. **(ii)** it minimizes communication bandwidth requirements, since transmitting compact metadata consumes far less bandwidth than streaming raw video. **(iii)** it enhances data privacy and storage efficiency, as sensitive visual content remains on the local device, reducing the need to store or transmit raw footage to centralized servers. Together, these advantages provide a solid foundation for developing real-time, scalable, and privacy-preserving MCVT systems. However, transforming this theoretical framework into a practical, city-scale solution still requires substantial progress in two key areas: (i) real-time algorithm design, and (ii) system-level scalability for deployment and operation.

(i) Real-time algorithm design. To ensure the real-time ability of our multi-camera tracking algorithm, three aspects need to be considered when designing the algorithm:

- **Edge-side optimization.** Each edge node performs the intra-camera process, including object detection and single-camera tracking, where several real-time pipelines have been proposed (Lashkov et al., 2023; Balamuralidhar et al., 2021). However, in the broader context of MCVT, reliable and robust cross-camera association additionally requires extracting discriminative appearance features for each vehicle. This feature extraction step removes the need to transmit raw images to the central server, improving privacy and bandwidth efficiency, but it is computationally intensive for embedded devices. Efficient deployment therefore depends on selecting lightweight yet effective models and fully exploiting heterogeneous on-board resources such as GPUs, DLAs, and CPU cores. Proper task allocation—for example, assigning lightweight network to DLA and assigning pre-/post-processing to certain CPU cores—can substantially improve the performance of whole system. In summary, achieving the right balance between representation quality, hardware utilization, and inference speed remains a core challenge for real-time operation.

- **Server-side coordination.** The inter-camera process is executed on the central server, where vehicle identities from different edge nodes are associated. Although previous researchers have proposed different methodologies (**?**Lin et al., 2025a), directly apply of those offline methods in real-world deployment may result imperfect performance. The metadata received from edge side may vary in quality and arrive asynchronously due to network latency or frame loss. Consequently, additional post-processing and temporal buffering are required to maintain stable performance. Therefore, the main challenge lies in adapting these offline algorithms to operate robustly under real-world conditions characterized by asynchronous, variable-quality data streams.

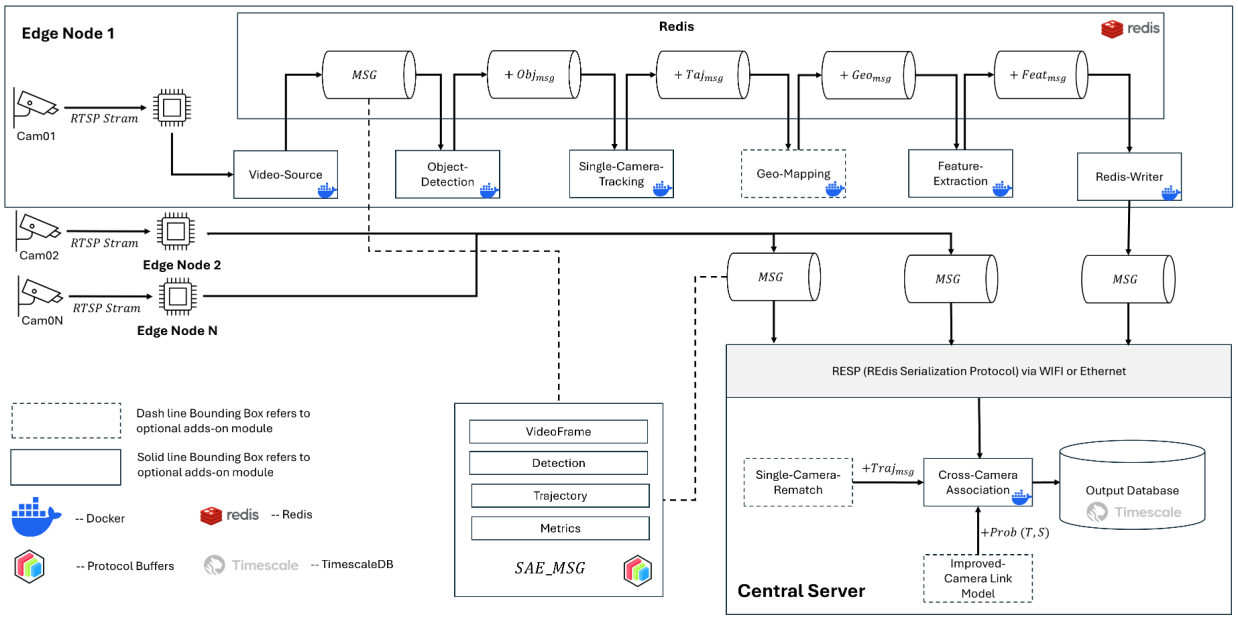


Figure 1: The workflow of SAE-MCVT framework. It contains two main blocks. (I) N edge node in the road network. Each edge node received raw rtsp stream from cameras, which then processed through multiple modules on computation unit, and only meta-data will be transmitted through RESP protocol to (II) central server, where meta trajectory data from different cameras will be further processed and associated to form complete cross camera trajectories.

(ii) Systems-level scalability for deployment and operations. To make our system scalable, some data-engineering challenges also requires to be investigated:

- **Data transmission.** In real-world, one requirement for a reliable system is good data transmission speed and data secure. How to maintain a good data transmission(speed and structured) is a hard requirement for a reliable system.
- **Deployability and operability.** City-scale systems need repeatable deployment, updates, and monitoring. How to find a way to define a scalable system is also very crucial for a success system.

To address the above challenges, we propose SAE-MCVT, an edge enhanced MCVT framework designed for real-time operation and city-scale deployment. To the best of author's knowledge, this is the first 'real-time' and scalable MCVT framework that provide solution for real-world MCVT deployment. The workflow of SAE-MCVT is proposed in Figure 1. And Our main contributions are as follows:

- **Real-time edge-enhanced MCVT pipeline.** We have evaluated the performance of our pipeline in our dataset which shows the SAE-MCVT embedded with NVIDIA Jetson AGX Orin can maintain its real-time performance in 4k 15FPS video with average IDF1 at 61.2%.
- **Open-sourced and standardized deployment package.** In our SAE-MCVT, every module are packed into a docker image which can be directly downloaded from DockerHub and can be directly used in different platforms.
- **Dataset and evaluation.** We have collected and open-sourced a comprehensive, high-resolution multi-camera vehicle tracking (MCVT) dataset named **RoundaboutHD**. It contains 40 minutes of fully annotated video footage recorded from 4 non-overlapping 4K cameras at 15 FPS, and provides a benchmark to evaluate our proposed MCVT framework.

The rest of paper is structured as follows: Section 2 provides a comprehensive literature review for the multi-camera tracking problem and researcher's effort to develop a real-time multi-camera tracking pipeline. Section 3

outlines the algorithm details for SAE-MCVT framework with evaluation on different dataset listed on Section 4. Finally, Section 5 wraps up the study by summarizing the main findings and proposing direction for future research.

2. Literature Review

As we already noted, the MCVT problem can be divided into two stages: (i) intra-camera process and (ii) inter-camera process. The task for intra-camera process is to detect several objects in a single frame, and then identify the temporal consistence across multiple frames from the same camera (Amosa et al., 2023). For the vehicle object detection task, current deep-learning based methods can be mainly classified into two categories: one-stage detectors, such as YOLO family(Tian et al., 2025; Varghese and Sambath, 2024), and two-stage detectors, such as Fast R-CNN Girshick (2015). Since 2020, one-stage detectors—particularly the YOLO family have become dominant in both academic research and real-world applications due to the balance of their speed and accuracy. Very recently, some researchers also tried to adopt the powerful pre-trained multi-model language language model(LLM) for the zero-shot object detection task (Zang et al., 2025; Bai et al., 2023). Due to the huge size and corresponding computational cost of LLM, YOLO family is still the first choice for edge-based real-world application. After acquired the object location, single camera tracking module is going to identify the temporal consistency of same objects. The single-camera tracking are mainly in two categories: deep-embeddings based and non-embedding based. For the deep embeddings based algorithm e.g DeepSORT (Wojke et al., 2017) and BotSort (Aharon et al., 2022), the similarity of objects across different frames depends both on the bounding-box (Bbox) Intersection over Union (IoU) and deep embeddings similarity. The calculation of deep embeddings improve the robustness of the tracker to handle occlusions or frame loss, but introduces more computational cost in the SCT process. For the non-embedding based methods, such as SORT (Bewley et al., 2016) and Bytetrack (Zhang et al., 2022). Those methods only relies on kalman-filter and IOU-based matching, which is lightweight and more suitable for resource-constraint edge-based real-world application.

For the inter-camera process, which also be known as the cross camera ReID process. This process originally come from the image-based ReID task, the aim is to associate images shoot from different cameras. This image-based ReID exclude the temporal information for single-camera process, which simplifies the process and avoid error from intra-camera process. Researchers focuses on extracting representative deep embeddings and different matching methods, some well-known researches includes PROVID (Liu et al., 2017), Git (Shen et al., 2023) and Going beyond real data Zheng et al. (2020). Those researches provides a solid foundation for the inter-camera MCVT process. After 2020, with computer vision technologies become more mature, researchers insights focuses more on the video-based vehicle ReID problem which is the MCVT. Compare to image-based ReID, the video-based ReID takes temporal information from each camera which provide richer information but also brings more challenges from the intra-camera process. One most obvious challenges is the computational resource increase for the feature extraction module as the expansion of temporal information brings the needs to process more information. While most of the feature extractor relies on heavy neural networks e.g. (Zhang et al., 2024) combines the result from four transformer network and three different ResNet to get more robust deep embedding features. Addition to heavy feature extractor, some studies (Yang et al., 2022) also introduce a very heavy SCT module to get more robust SCT results, e.g. (Li et al., 2022) combines the result from three different SCT modules to get a more robust SCT performance, which scarifies the computational efficiency. And (Yang et al., 2022)'s SCT module takes a very heavy deep embeddings as input, combine multiple SCT algorithm together and also introduced an offline re-link module to make the SCT module very redundant. Moreover, some studies (Liu et al., 2021; Wu et al., 2021) leverage manually labeled zones to capture spatio-temporal relationships within and across adjacent cameras. While this approach performs well in small-scale testing environments, it becomes impractical for city-scale deployment. The reliance on human-labeled, predefined zones introduces a significant workload, and any change in camera position or orientation—whether intentional or accidental—necessitates re-labeling, making such methods infeasible for real-world applications. Overall, works mentioned above provide a strong theoretical foundation for MCVT problem and also point out the main challenge for deploying MCVT to real-world, which is the balance between accuracy and speed.

Although no existing system fully addresses the real-world Multi-Camera Vehicle Tracking (MCVT) problem, several studies have made progress toward enabling real-time or deployable vision-based Intelligent Transportation Systems (ITS). For instance, Zhang and Izquierdo (2019) proposed a real-time multi-camera tracking framework that processes video data frame by frame instead of waiting for each batch to complete, achieving real-time performance

on the DukeMTMC dataset (Ristani et al., 2016a). Building on this direction, Zhuang et al. (2024) developed a real-time MCVT framework that takes continuous video streams as input and demonstrates real-time performance on both the CityFlow dataset (Tang et al., 2019) and their custom dataset. However, the real-time capability of these systems heavily depends on high-end hardware, and performance quickly becomes a bottleneck as the number of cameras increases. To mitigate this limitation, several studies (Liu et al., 2023; Ke et al., 2020) have explored the use of edge devices to distribute computational workloads for practical ITS deployment. Notably, both of these works have been successfully deployed in real-world environments. Although they are not directly focused on MCVT, their success demonstrates the effectiveness of edge computing in real-world, vision-based ITS applications. Another notable work is that of Yang et al. (2023) from the University of Washington, which employs edge computing for real-world vehicle re-identification (ReID). Due to several constraints, however, this system operates only at the clip level rather than the trajectory level. The work most closely related to ours is Gaikwad and Karmakar (2021), which applies edge computing to multi-camera pedestrian tracking to achieve scalable and real-time performance. Nonetheless, in real traffic networks, pedestrians exhibit much lower speeds, smaller activity ranges, and smaller scales compared to vehicles, making vehicle tracking a far more challenging problem.

3. SAE-MCVT Framework Methodology

3.1. SAE-MCVT Overall Structure

To solve the demand of developing a real-time and scalable MCVT system, we have proposed the SAE-MCVT framework, with graphic illustration shown in fig. 1. This framework is consisted by two parts: (i) edge node (ii) central server. Each edge node integrates a camera and a computing unit—implemented using an NVIDIA Jetson AGX Orin. The camera continuously transmits video streams via RTSP to the computing unit, where the data is processed through a series of containerized modules, including *video-source*, *object-detection*, *single-camera-tracking*, *geo-mapping*, *feature-extraction*, *redis-writer*. Through these modules, the system generates detailed trajectory information for all detected vehicles within a single camera view. The extracted trajectory metadata is then transmitted to the central server through RESP over Wi-Fi or the Internet. On the central workstation, a *single-camera re-matching module* first refines fragmented trajectories caused by occlusions, frame loss, or tracking interruptions. This is followed by a pre-trained, self-supervised *camera link model*, which estimates spatial–temporal relationships between adjacent cameras. Using this learned spatial–temporal context, the system narrows the search space for the final *cross-camera association*, resulting in complete multi-camera trajectories. Finally, all reconstructed trajectories are stored in the output database for downstream analysis and long-term archival.

To enable practical, large-scale deployment, the proposed framework is implemented on top of the open-source SAE-Engine (starwit, 2025), developed by our industrial partner. The SAE-Engine provides a set of pre-built modules for common computer-vision tasks and templates for customizing new ones. Its key advantage lies in integrating several mature software platforms—Docker, Redis, Google Protocol Buffers, and TimescaleDB—to standardize development and ensure scalability. Each component in the MCVT pipeline is containerized using Docker for modular and reproducible deployment. Data are serialized in predefined Protocol Buffer schemas to maintain a consistent and structured format. The structured metadata are transmitted through Redis, which provides low-latency, reliable communication among modules, and the final outputs are stored in TimescaleDB to support efficient querying and long-term data management.

3.2. Edge Node Design

3.2.1. Raw Video Pre-Process

In real-world deployments, video data are typically streamed in real time and may suffer from frame loss due to network latency or transmission instability. To facilitate downstream processing and data management, we extract several key attributes from each frame, including the source ID, timestamp, frame dimensions, and base64-encoded frame data. These elements are then serialized for efficient storage and subsequent use.

3.2.2. Real-time Object Detection

3.2.2.1. YOLO detector. In recent years, YOLO family have become dominant in both academic research and real-world applications due to their speed and accuracy.Among its variants, YOLOv8 (Varghese and Sambath, 2024) is a conventional CNN-based detector, whereas YOLOv11 (Khanam and Hussain, 2024) and YOLOv12 (Tian et al., 2025) employ CNN backbones augmented with attention mechanisms to improve detection performance. However, for our specific application, all YOLO models are pre-trained on general object detection coco dataset (Lin et al., 2014), which contains limited vehicle imagery from overhead surveillance viewpoints. In our inspections, this led to frequent misses for overhead perspectives and for small, distant vehicles. To address this gap, we retrained YOLO models on the RoundaboutHD object-detection dataset (Lin et al., 2025b). After comparing latency and accuracy across candidates, we selected the retrained YOLOv11n and deployed it with a TensorRT backend in our pipeline.

3.2.2.2. Motion Detector. In real-world deployments, traffic scenes often contain extended periods with no vehicle activity, especially during late-night hours. Running the full detection and tracking pipeline during such intervals is unnecessary and wastes computational resources on the edge device. To address this, we introduce a lightweight motion detector placed before the YOLO module to determine whether meaningful motion is present. This module is implemented using OpenCV's Gaussian Mixture-based Background/Foreground Segmentation (MOG2), which estimates the number of moving pixels in each frame. If the number of foreground pixels exceeds a threshold T_{pixel}, the YOLO-based object detector is activated; otherwise, all downstream processing is skipped for that frame.

3.2.3. Single Camera Tracking

In single camera tracking (SCT) module, object bounding boxes got from the previous object detection module in each frame are processed to get unified ID across different frames. Given the limited computational resources on the edge devices, we focus on non-deep-embedding-based tracking methods, in particular ByteTrack (Zhang et al., 2022) and OCSORT (Cao et al., 2023), both of which achieve strong performance on the MOT20 benchmark (Dendorfer et al., 2020). Based on comparative evaluations under our target deployment conditions, we select ByteTrack as the SCT baseline for the proposed SAE-MCVT framework.

3.2.4. Geo-Mapping

The geo-mapping module converts vehicle locations from image coordinates to real-world GPS coordinates, which is not essential for MCVT task but allow many further practical applications such as traffic-flow digital twins and suspect vehicle retrieval. The process can be expressed as:

$$GPS_{lat,lon} = Cam(x_{pixel}, y_{pixel}, height) \tag{1}$$

Where *lat* and *lon* are the GPS latitude and longitude of vehicle, *Cam* is the camera transformation model from (Gerum et al., 2017), x_{pixel} and y_{pixel} are the vehicle center pixel location with *height*] to be the estimate gravity height of each vehicles.

3.2.5. Feature Extraction

3.2.5.1. Dynamic Workload Adjustment Scheme. Feature extraction typically relies on deep neural networks to generate representative embeddings from pre-tracked trajectories. However, performing feature extraction frame-by-frame is computationally expensive for resource-constrained edge devices, as it requires huge GPU capacity. Instead of processing every frame, our pipeline buffers the incoming trajectory data and performs feature extraction only after a trajectory is completed within a camera's field of view. Each completed trajectory is treated as a task, and all tasks are maintained in a first-in, first-out (FIFO) queue. We consider the feature-extraction process to be real-time if the system can process tasks at a faster rate than new tasks arrive. Although this design introduces slight latency, it does not affect the overall pipeline for non-overlapping FOV MCVT because there is a time gap for vehicles to exit one camera view and appear in the next.

To formalize this process, we define:

$$DataVolume_{min} = \sum_{i=1}^{n} NumFrame_i \tag{2}$$

Where $DataVolume_{min}$ is the total number of frames to be processed per minute, n is number of incoming tasks and $NumFrame_i$ is the number of frames included in trajectory i. To further reduce computation, we subsample each trajectory using:

$$NumFrame_i = \frac{TrackletLength_i}{K} \tag{3}$$

where K is a hyperparameter controlling the subsampling rate—only one frame out of every K frames is used for feature extraction.

To ensure stable performance under varying traffic conditions, we design a dynamic adjustment scheme with a predefined task-queue threshold T. The number of incoming tasks depends on the traffic density, which is not controllable, but the computational load can be regulated by adjusting K. When the queue length exceeds T, the system increases K to reduce $DataVolume_{min}$; when the queue length falls below T, K is decreased to improve feature quality. This adaptive mechanism prevents task overflow while maintaining acceptable latency.

3.2.5.2. Implementation Details. For the feature extraction model, we adopt ResNet-50 (He et al., 2016) as the backbone due to its favorable balance between accuracy and inference speed on edge hardware. The network is pre-trained on the RoundaboutHD image-based ReID dataset, ensuring that the extracted embeddings are well aligned with the overhead surveillance domain.

For each completed trajectory, the deep feature extractor outputs an $[N * 2048]$ matrix, where N is number of frames of one trajectory. Followed by that, We apply L2 normalization to get the final $[1 * 2048]$ feature vector. The process is as follow:

$$\hat{\mathbf{f}} = \frac{\sum_{i=1}^{N} c_i \mathbf{f}_i}{\left\| \sum_{i=1}^{N} c_i \mathbf{f}_i \right\|_2} \tag{4}$$

where $\mathbf{f}$ is the final $[1*2048]$ feature vector, f_i is the feature vector of each cropped detection and c_i is the corresponding confidence score for each candidate detection.

Finally, all relevant metadata—including bounding boxes, GPS coordinates, and normalized deep features—are transmitted from each edge node to the central server for inter-camera association.

3.3. Central Server Design

The central server received serialized SAE_MSG from multiple different edge node through the REdis Serialization Protocol via WIFI or Ethernet for the cross-camera association. And save the result in a database for downstream applications.

3.3.1. Tracklet Remerge

In real-world deployments, single-camera tracking (SCT) can be affected by lighting changes, occlusions, irregular vehicle motion, and occasional frame loss. These factors often cause identity switches when lightweight, non-embedding-based SCT algorithms are used on edge devices. Although embedding-based SCT approaches can reduce such errors, performing feature extraction before SCT would require frame-by-frame inference, which is computationally infeasible for resource-constrained edge hardware.

To mitigate these issues without increasing the edge-side burden, our pipeline incorporates an optional tracklet re-merging module executed on the central server. This module refines the initial SCT results by rejoining fragmented tracklets that are likely to belong to the same vehicle. Users may enable or disable this module depending on the computing capacity of their central server. The re-merging logic is rule-based and is summarized as follows:

$$Merge(Tracklet_A, Tracklet_B) = \begin{cases} 1, & \text{if } \begin{cases} 0 < t_B^{\text{start}} - t_A^{\text{end}} < T_{\text{th}}, \\ \left\| \mathbf{b}_B^{\text{start}} - \mathbf{b}_A^{\text{end}} \right\|_2 < D_{\text{th}}, \\ 1 - \max\left(\cos_\text{sim}(\bar{\mathbf{f}}_A, \bar{\mathbf{f}}_B),\ \cos_\text{sim}(\mathbf{f}_A^{\text{end}}, \mathbf{f}_B^{\text{start}}) \right) < F_{\text{th}} \end{cases} \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

In Eq. (5), $\text{Merge}(\cdot) \in \{0, 1\}$ indicates whether two candidate tracklets—assuming Tracklet_A precedes Tracklet_B—should be merged. The variables t_A^{end} and t_B^{start} represent the end time of Tracklet_A and the start time of Tracklet_B,

with T_{th} denoting the maximum allowed temporal gap. The vectors $\mathbf{b}_A^{\text{end}}, \mathbf{b}_B^{\text{start}} \in \mathbb{R}^4$ correspond to the bounding boxes at the end of Tracklet$_A$ and the start of Tracklet$_B$, and D_{th} is the spatial distance threshold. The appearance descriptors $\bar{\mathbf{f}}_A, \bar{\mathbf{f}}_B \in \mathbb{R}^d$ are the L2-normalized averages of the per-frame features within each tracklet, while $\mathbf{f}_A^{\text{end}}$ and $\mathbf{f}_B^{\text{start}}$ denote the features of the last detection of Tracklet$_A$ and the first detection of Tracklet$_B$, respectively. The function cos_sim$(\cdot,\cdot)$ computes the cosine similarity between two feature vectors, and F_{th} is the threshold on appearance-based distance.

3.3.2. Self-Supervised Camera Link Model

For the MCVT problem, many previous studies (e.g., (Hsu et al., 2019)) introduce spatial–temporal constraints based on road layout and travel-time statistics to filter candidate trajectories between adjacent cameras. While effective, manually specifying these constraints is labor-intensive and impractical for large-scale deployments. Inspired by (Lin et al., 2025c), we adopt a *self-supervised camera link model* that automatically learns these spatial–temporal relationships offline without requiring human annotations. The model consists of three major steps:

(1) Identifying entry-exit under single camera scene. For each camera i, we define the set of zones as

$$\mathcal{Z}_i = \{\mathbf{z}_{i,k} \,|\, k = 1, \ldots, N_i\}, \tag{6}$$

$$\mathcal{Z}_i^{\text{entry}} = \{\mathbf{z}_{i,m}^{\text{entry}} \,|\, m = 1, \ldots, N_i^{\text{entry}}\} \subseteq \mathcal{Z}_i, \tag{7}$$

$$\mathcal{Z}_i^{\text{exit}} = \{\mathbf{z}_{i,n}^{\text{exit}} \,|\, n = 1, \ldots, N_i^{\text{exit}}\} \subseteq \mathcal{Z}_i, \tag{8}$$

These zones are obtained by clustering the first and last center-point locations of all single-camera trajectories, which reliably correspond to entry and exit regions in a given scene.

(2) Entry-exit zone pairing across adjacent camera. Once zones are defined for each camera, the next step is to identify zone correspondences between adjacent camera pairs. For cameras i and j with N_i^{exit} exit zones and N_j^{entry} entry zones, we compute a pairing score:

$$s_{i,j}\left(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}\right) = \text{ZonePair}\left(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}\right), \tag{9}$$

for all (m, n) pairs. The best-matching pair is selected via

$$(\hat{m}, \hat{n}) = \arg\max_{m,n} s_{i,j}\left(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}\right), m \in \{1, \ldots, N_i^{\text{exit}}\},\ n \in \{1, \ldots, N_j^{\text{entry}}\}. \tag{10}$$

The definition of the ZonePair function is provided in (Lin et al., 2025c).

(3) Camera link transition time estimation. For each selected entry–exit zone pair (i, j), we estimate the distribution of vehicle travel times using Gaussian kernel density estimation (KDE). Let $\{\tau_{i,j}^{(\ell)}\}_{\ell=1}^{L_{i,j}}$ denote the observed travel times. The continuous transition-time density is estimated as

$$\hat{p}_{i,j}(t) = \frac{1}{L_{i,j} h} \sum_{\ell=1}^{N_{i,j}} \mathcal{K}\left(\frac{t - \tau_{i,j}^{(\ell)}}{h}\right), \tag{11}$$

where $h > 0$ is the bandwidth and $\mathcal{K}(\cdot)$ is the Gaussian kernel

$$\mathcal{K}(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right). \tag{12}$$

Thus, $\hat{p}_{i,j}(t)$ represents the estimated probability density of transition times between the paired zones.

Using these learned spatial–temporal constraints, only trajectory candidates whose exit–entry zones match $(\hat{m}, \hat{n})$ and whose travel time satisfies $\hat{p}_{i,j}(t) < T_{\text{prob}}$ are forwarded to the cross-camera association stage. This significantly reduces the search space while preserving valid vehicle matches across cameras.

3.3.3. Cross-Camera Association

In the cross-camera association stage, the central server continuously receives metadata from all edge nodes. Ideally, a cross-camera association procedure should be executed every time a new tracklet arrives, ensuring that the global trajectory set is always up to date. However, triggering association on every arrival would impose significant computational load and latency on the central server. To balance responsiveness and efficiency, we perform association periodically, every α seconds, and maintain a sliding buffer that stores the most recent β seconds of tracklets for matching.

During each association cycle, we process adjacent camera pairs. For camera pairs $i\&j$, we construct a cost matrix between all candidate tracklets from the two cameras. The pairwise cost between tracklets T_i and T_j is defined as:

$$\text{Cost}(T_i, T_j) = \delta \cos(T_i, T_j) - \epsilon \hat{p}_{i,j}\Big(\|T_i - T_j\|\Big), \tag{13}$$

where $\cos(T_i, T_j)$ denotes the appearance similarity between the two tracklets, $\hat{p}_{i,j}(\cdot)$ is the transition-time probability density obtained from the camera-link model, and δ and ϵ are hyperparameters controlling the contributions of appearance and temporal consistency.

The full cost matrix between tracklets from cameras i and j is then

$$\mathbf{C} = \begin{bmatrix} \text{Cost}(T_1, T'_1) & \cdots & \text{Cost}(T_1, T'_m) \\ \vdots & \ddots & \vdots \\ \text{Cost}(T_n, T'_1) & \cdots & \text{Cost}(T_n, T'_m) \end{bmatrix}, \tag{14}$$

where $\{T_1, \ldots, T_n\}$ and $\{T'_1, \ldots, T'_m\}$ denote the tracklets from cameras i and j, respectively.

For each adjacent camera pair, we apply a greedy matching algorithm to the cost matrix to merge tracklets into unified trajectories. The results from all camera pairs are then consolidated and written to the database, where they are integrated with historical trajectory information to obtain complete multi-camera trajectories.

4. Experiment

4.1. Dataset

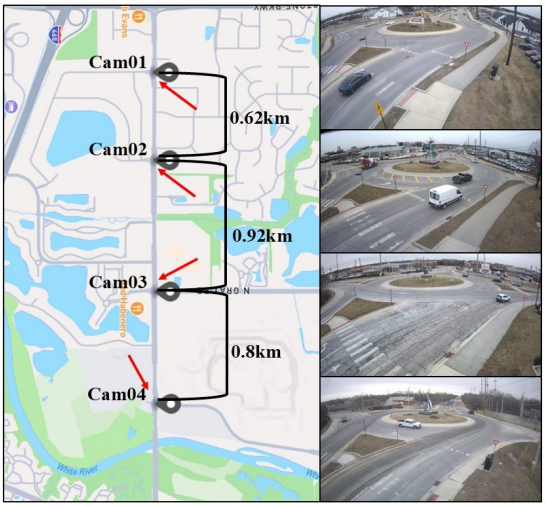
4.1.1. CityFlow

CityFlow (Tang et al., 2019) is one of the earliest and most representative open-source MCVT datasets, released by NVIDIA Research in 2019. It contains approximately 3.25 hours of 960p traffic video recorded from 46 cameras across 10 intersections in a medium-sized U.S. city, covering roughly 2.5 km of roadway. In this study, we use the training and validation splits (40 cameras) to pre-train our feature extractor, and the 6-camera test split to evaluate the performance of the proposed SAE-MCVT pipeline.

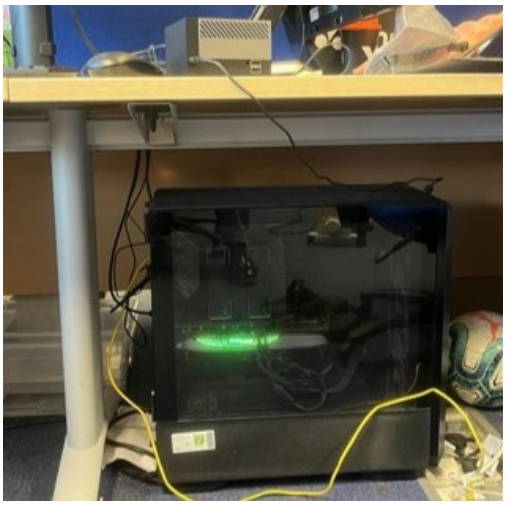
4.1.2. RoundaboutHD

RoundaboutHD (Lin et al., 2025b) is a recently introduced, high-quality open-source MCVT dataset captured at a real-world roundabout, it's layout is shown in fig. 2a. It contains 40 minutes of fully annotated, non-overlapping multi-camera video captured from four 4K cameras at 15 FPS. The dataset introduces several challenges for MCVT, including: (1) nonlinear vehicle trajectories, (2) frequent occlusions caused by infrastructure such as the central statue, and (3) multiple exits and intersections across camera views. Until the submission, the author haven't seen any published works have fully addressed these challenges.

Compared with CityFlow, RoundaboutHD provides longer recordings and significantly higher resolution, making it highly suitable for our real-time, high-precision deployment scenario. In addition to the main MCVT dataset, RoundaboutHD offers several task-specific subsets for object detection, single-camera tracking, and image-based vehicle ReID. In our experiments, we use the object-detection subset to fine-tune the YOLO detector and the ReID subset to train our appearance feature extractor. As a result, RoundaboutHD serves as the primary dataset for evaluating the SAE-MCVT framework.



(a) Layout of RoundaboutHD dataset.



(b) The hardware setting

4.2. Implementation Details

4.2.1. Hardware settings

Due to hardware availability constraints, the SAE-MCVT framework is evaluated in a laboratory environment using a single NVIDIA Jetson AGX Orin 64 GB module as the edge computing device, and a workstation equipped with an Intel 13th Gen i7-13700KF CPU and an NVIDIA GeForce RTX 4070 Ti GPU as the central server, running Ubuntu 20.04.4 LTS. The edge node and the central server are connected via a wired Ethernet link.

Because only one Jetson device is available, we simulate a multi-camera setup by running each edge node configuration independently, recording the resulting SAE_MSG output, and replaying multiple streams simultaneously to emulate multi-node operation. The overall hardware configuration is illustrated in fig. 2b

4.2.2. Hyperparameter settings

- **Object Detection.** We follow the standard YOLO inference pipeline. The confidence threshold is set to 0.35 and the IoU threshold for non-maximum suppression is 0.40. The image inference size is configured as 1120×620 to improve detection of small, distant vehicles. All remaining settings follow the default YOLO configuration. The motion detector threshold T_{pixel} is set to 300.
- **Single-Camera Tracking.** We adopt the default parameter settings from BoxMOT (Broström, 2025) for the single-camera tracking module.
- **Geo-Mapping.** The vehicle center-of-gravity height is assumed to be 0.5 m for all vehicles when applying the camera transformation model.
- **Feature Extraction.** For the dynamic workload adjustment scheme, the subsampling rate is initialized as $K = 5$, and the task-queue threshold is set to $T = 10$.
- **Tracklet Re-Merge.** For refining fragmented single-camera tracklets, we use a temporal threshold of $T_{\text{th}} = 4\,\text{s}$ (60 frames), a spatial threshold of $D_{\text{th}} = 0.25$ of the image width, and a feature-similarity threshold of $F_{\text{th}} = 0.2$.
- **Self-Supervised Camera Link Model.** The bandwidth parameter for the KDE-based transition-time estimation is set to 5.0.
- **Cross-Camera Association.** The association process is executed every $\alpha = 200\,\text{s}$ (3000 frames), using a sliding buffer of $\beta = 300\,\text{s}$ (4500 frames). The cost-function weights are set to $\delta = 1$ and $\epsilon = 5$, and the greedy-matching distance threshold is 0.4.

4.3. SAE-MCVT framework performance evaluation

4.3.1. Real-Time ability Evaluation

Real-time capability is a critical requirement for multi-camera tracking systems. To assess the real-time performance of the proposed SAE-MCVT framework, we measure the per-frame processing time (in milliseconds) and verify that the system sustains the required FPS under the RoundaboutHD settings. The results are summarized in Table 1.

Table 1: Real-Time Performance Evaluation for SAE-MCVT.

	RoundaboutHD	CityFlow
Object Detection	15ms per frame	\
Single Camera Tracking	3ms per frame	\
Geo-Mapping	3ms per frame	\
Feature Extraction	No queue Overflow	\
Cross-Camera Association	No queue Overflow	\

As shown in Table 1, the proposed framework achieves stable real-time performance under a 2K, 15 FPS, four-camera configuration on the RoundaboutHD dataset. No queue overflow is observed in the feature extraction or cross-camera association modules, indicating that the dynamic workload adjustment mechanism functions effectively.

4.3.2. Performance ability Evaluation

To evaluate MCVT accuracy, we report the identification-based metrics IDF1, IDP, and IDR (Ristani et al., 2016b) on the RoundaboutHD dataset. The quantitative results are presented in Table 2.

Dataset	**IDF1**	**IDP**	**IDR**
RoundaboutHD	61.96	91.02	46.96

Table 2: Vehicle Contextual Information Based Performance Evaluation on RoundaboutHD.

The results demonstrate that the SAE-MCVT framework achieves strong performance on the RoundaboutHD dataset and successfully handles the challenges introduced by complex trajectories, frequent occlusions, and multi-exit camera topology. The high IDP score indicates that the system produces highly precise associations, while the relatively lower IDR suggests that some correct matches are missed. This is likely due to conservative hyperparameter settings; relaxing certain thresholds may improve recall without significantly affecting precision.

4.3.3. Self-Supervised Camera Link Model

To evaluate the effectiveness of the proposed self-supervised camera link model, we compare the estimated transition-time distribution with the ground-truth transition-time histogram derived from annotated trajectories. The results shows our estimated transition-time distribution closely aligns with the ground-truth distribution, demonstrating that the camera link model successfully captures the underlying spatial–temporal relationships between adjacent camera views. The illustration is shown in Figure 3.

4.4. Additional Experiments (Ongoing)

The above evaluation provides an initial validation of the real-time capability and tracking performance of the SAE-MCVT framework. To further provide a comprehensive assessment, several additional experiments are planned and are currently in progress:

- Ablation studies on the SAE-MCVT framework components.
- Real-time and accuracy evaluation on the CityFlow dataset.

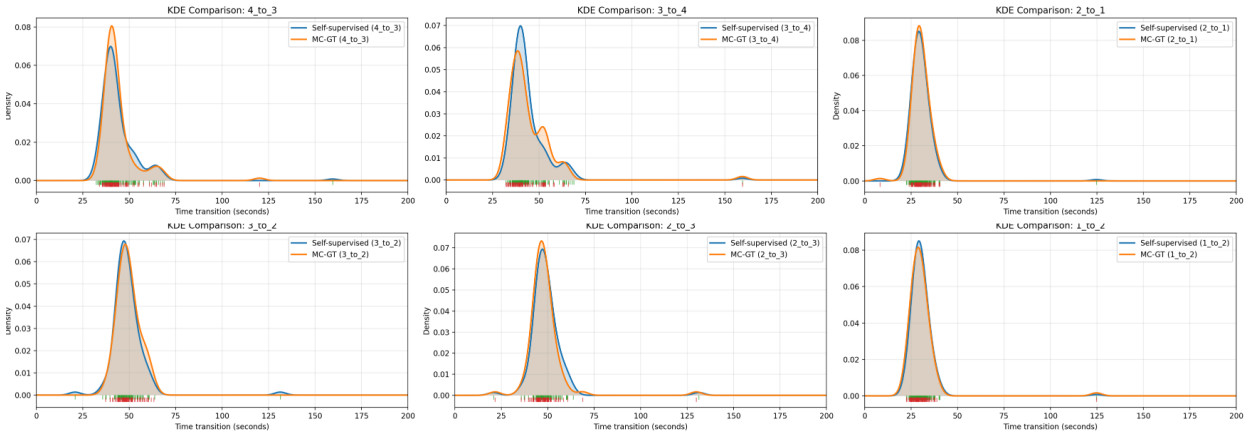


Figure 3: Comparison between ground-truth and estimated transition-time distributions for the self-supervised camera link model.

- Edge-side performance profiling on RoundaboutHD.
- Scalability verification under increased numbers of edge nodes.
- Preliminary real-world deployment tests in Barcelona or Berlin.

These experiments will be included in the next revision of the manuscript, and a revised version of the paper will be released on arXiv prior to submission to *Transportation Research Part C*.

5. Conclusion

This work presents the SAE-MCVT framework, the first multi-camera vehicle tracking system designed to be both *real-time* and *scalable* for practical deployment. The framework provides a standardized, modular pipeline that aligns with common industrial standards while addressing the computational constraints of edge–cloud architectures. Through extensive experimentation, SAE-MCVT demonstrates the ability to operate in real time on resource-limited edge devices. This is achieved through the use of a fine-tuned YOLO-nano detector, a lightweight non-embedding-based SCT module, and a dynamic workload adjustment scheme that adapts to varying traffic conditions. On the central server, the tracklet re-merging mechanism and the self-supervised camera link model further enhance cross-camera consistency and improve association accuracy. The proposed pipeline achieves an IDF1 score of 61.07 on the challenging RoundaboutHD benchmark, illustrating its effectiveness under complex real-world conditions. Several additional experiments—including ablation studies, large-scale performance evaluation, and real-world deployment tests—are currently underway. The authors believe that these forthcoming results will further demonstrate the advantages and robustness of the proposed approach. Overall, the SAE-MCVT framework offers a practical and scalable solution for multi-camera vehicle tracking and represents a significant step toward real-world, city-scale MCVT applications.

Acknowledgment

Yuqiang Lin and Sam Lockyer are supported by a scholarship from the EPSRC Centre for Doctoral Training in Advanced Automotive Propulsion Systems (AAPS) under project EP/S023364/1. The authors would also like to express their gratitude to the residents and city council of Carmel, Indiana, USA, for their support of this research.

References

Aharon, N., Orfaig, R., Bobrovsky, B.Z., 2022. Bot-sort: Robust associations multi-pedestrian tracking. arXiv preprint arXiv:2206.14651 .

Amosa, T.I., Sebastian, P., Izhar, L.I., Ibrahim, O., Ayinla, L.S., Bahashwan, A.A., Bala, A., Samaila, Y.A., 2023. Multi-camera multi-object tracking: A review of current trends and future advances. Neurocomputing 552, 126558.

Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al., 2023. Qwen technical report. arXiv preprint arXiv:2309.16609 .

Balamuralidhar, N., Tilon, S., Nex, F., 2021. Multeye: Monitoring system for real-time vehicle detection, tracking and speed estimation from uav imagery on edge-computing platforms. Remote sensing 13, 573.

Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B., 2016. Simple online and realtime tracking, in: 2016 IEEE international conference on image processing (ICIP), Ieee. pp. 3464–3468.

Broström, M., 2025. BoxMOT: Pluggable sota multi-object tracking modules for segmentation, object detection, and pose estimation. `https://github.com/mikel-brostrom/boxmot`. Commit version used: v13.0.17 (July 2025).

Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K., 2023. Observation-centric sort: Rethinking sort for robust multi-object tracking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 9686–9696.

Dendorfer, P., Rezatofighi, H., Milan, A., Shi, J., Cremers, D., Reid, I., Roth, S., Schindler, K., Leal-Taixé, L., 2020. Mot20: A benchmark for multi object tracking in crowded scenes. arXiv preprint arXiv:2003.09003 .

Gaikwad, B., Karmakar, A., 2021. Smart surveillance system for real-time multi-person multi-camera tracking at the edge. Journal of real-time image processing 18, 1993–2007.

Gerum, R., Richter, S., Winterl, A., Fabry, B., Zitterbart, D., 2017. Cameratransform: a scientific python package for perspective camera corrections. arXiv preprint arXiv:1712.07438 .

Girshick, R., 2015. Fast r-cnn, in: Proceedings of the IEEE international conference on computer vision, pp. 1440–1448.

Gong, T., Zhu, L., Yu, F.R., Tang, T., 2023. Edge intelligence in intelligent transportation systems: A survey. IEEE Transactions on Intelligent Transportation Systems 24, 8919–8944.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778.

Holla, A., MM, M.P., Verma, U., Pai, R.M., 2025. Vehicle re-identification and tracking: Algorithmic approach, challenges and future directions. IEEE Open Journal of Intelligent Transportation Systems .

Hsu, H.M., Huang, T.W., Wang, G., Cai, J., Lei, Z., Hwang, J.N., 2019. Multi-camera tracking of vehicles based on deep features re-id and trajectory-based camera link models., in: CVPR workshops, pp. 416–424.

Ke, R., Liu, C., Yang, H., Sun, W., Wang, Y., 2022. Real-time traffic and road surveillance with parallel edge intelligence. IEEE Journal of Radio Frequency Identification 6, 693–696.

Ke, R., Zhuang, Y., Pu, Z., Wang, Y., 2020. A smart, efficient, and reliable parking surveillance system with edge artificial intelligence on iot devices. IEEE Transactions on Intelligent Transportation Systems 22, 4962–4974.

Khanam, R., Hussain, M., 2024. Yolov11: An overview of the key architectural enhancements. arXiv preprint arXiv:2410.17725 .

Lashkov, I., Yuan, R., Zhang, G., 2023. Edge-computing-empowered vehicle tracking and speed estimation against strong image vibrations using surveillance monocular camera. IEEE Transactions on Intelligent Transportation Systems 24, 13486–13502.

Li, F., Wang, Z., Nie, D., Zhang, S., Jiang, X., Zhao, X., Hu, P., 2022. Multi-camera vehicle tracking system for ai city challenge 2022, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 3265–3273.

Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L., 2014. Microsoft coco: Common objects in context, in: Computer vision–ECCV 2014: 13th European conference, zurich, Switzerland, September 6-12, 2014, proceedings, part v 13, Springer. pp. 740–755.

Lin, Y., Lockyer, S., Evans, A., Zarbock, M., Zhang, N., 2025a. Ablation study for multicamera vehicle tracking using cityflow dataset, in: Seventeenth International Conference on Machine Vision (ICMV 2024), SPIE. pp. 246–253.

Lin, Y., Lockyer, S., Sui, M., Gan, L., Stanek, F., Zarbock, M., Li, W., Evans, A., Zhang, N., 2025b. Roundabouthd: High-resolution real-world urban environment benchmark for multi-camera vehicle tracking. arXiv preprint arXiv:2507.08729 .

Lin, Y., Lockyer, S., Zhang, N., 2025c. City-scale multi-camera vehicle tracking system with improved self-supervised camera link model. URL: https://arxiv.org/abs/2405.11345, arXiv:2405.11345.

Liu, C., Yang, H., Zhu, M., Wang, F., Vaa, T., Wang, Y., 2023. Real-time multi-task environmental perception system for traffic safety empowered by edge artificial intelligence. IEEE Transactions on Intelligent Transportation Systems 25, 517–531.

Liu, C., Zhang, Y., Luo, H., Tang, J., Chen, W., Xu, X., Wang, F., Li, H., Shen, Y.D., 2021. City-scale multi-camera vehicle tracking guided by crossroad zones, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4129–4137.

Liu, X., Liu, W., Mei, T., Ma, H., 2017. Provid: Progressive and multimodal vehicle reidentification for large-scale urban surveillance. IEEE Transactions on Multimedia 20, 645–658.

Qian, Y., Yu, L., Liu, W., Hauptmann, A.G., 2020. Electricity: An efficient multi-camera vehicle tracking system for intelligent city, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, pp. 588–589.

Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C., 2016a. Performance measures and a data set for multi-target, multi-camera tracking, in: European conference on computer vision, Springer. pp. 17–35.

Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C., 2016b. Performance measures and a data set for multi-target, multi-camera tracking, in: European conference on computer vision, Springer. pp. 17–35.

Shen, F., Xie, Y., Zhu, J., Zhu, X., Zeng, H., 2023. Git: Graph interactive transformer for vehicle re-identification. IEEE Transactions on Image Processing 32, 1039–1051.

starwit, 2025. Starwit awareness engine: A flexible computer-vision pipeline that connects to video source, detects, tracks and geo-maps objects. https://github.com/starwit/starwit-awareness-engine.

Tang, Z., Naphade, M., Liu, M.Y., Yang, X., Birchfield, S., Wang, S., Kumar, R., Anastasiu, D., Hwang, J.N., 2019. Cityflow: A city-scale benchmark for multi-target multi-camera vehicle tracking and re-identification, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 8797–8806.

Tian, Y., Ye, Q., Doermann, D., 2025. Yolov12: Attention-centric real-time object detectors. arXiv preprint arXiv:2502.12524 .

Varghese, R., Sambath, M., 2024. Yolov8: A novel object detection algorithm with enhanced performance and robustness, in: 2024 International conference on advances in data engineering and intelligent computing systems (ADICS), IEEE. pp. 1–6.

Wojke, N., Bewley, A., Paulus, D., 2017. Simple online and realtime tracking with a deep association metric, in: 2017 IEEE international conference on image processing (ICIP), IEEE. pp. 3645–3649.

Wu, M., Qian, Y., Wang, C., Yang, M., 2021. A multi-camera vehicle tracking system based on city-scale vehicle re-id and spatial-temporal information, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 4077–4086.

Yang, H.F., Cai, J., Liu, C., Ke, R., Wang, Y., 2023. Cooperative multi-camera vehicle tracking and traffic surveillance with edge artificial intelligence and representation learning. Transportation research part C: emerging technologies 148, 103982.

Yang, X., Ye, J., Lu, J., Gong, C., Jiang, M., Lin, X., Zhang, W., Tan, X., Li, Y., Ye, X., et al., 2022. Box-grained reranking matching for multi-camera multi-target tracking, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 3096–3106.

Zang, Y., Li, W., Han, J., Zhou, K., Loy, C.C., 2025. Contextual object detection with multimodal large language models. International Journal of Computer Vision 133, 825–843.

Zhang, X., Izquierdo, E., 2019. Real-time multi-target multi-camera tracking with spatial-temporal information. 2019 IEEE Visual Communications and Image Processing (VCIP) , 1–4.

Zhang, X., Yu, H., Qin, Y., Zhou, X., Chan, S., 2024. Video-based multi-camera vehicle tracking via appearance-parsing spatio-temporal trajectory matching network. IEEE Transactions on Circuits and Systems for Video Technology 34, 10077–10091.

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X., 2022. Bytetrack: Multi-object tracking by associating every detection box, in: European conference on computer vision, Springer. pp. 1–21.

Zheng, Z., Jiang, M., Wang, Z., Wang, J., Bai, Z., Zhang, X., Yu, X., Tan, X., Yang, Y., Wen, S., et al., 2020. Going beyond real data: A robust visual representation for vehicle re-identification, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, pp. 598–599.

Zhou, X., Ke, R., Yang, H., Liu, C., 2021. When intelligent transportation systems sensing meets edge computing: Vision and challenges. Applied Sciences 11, 9680.

Zhuang, H., Chen, X., Lan, J., Wang, Y., Lu, Z., Wang, C., Yang, M., 2024. A real-time framework of multi-camera vehicle tracking system, in: 2024 IEEE International Conference on Real-time Computing and Robotics (RCAR), IEEE. pp. 149–154.